

Họ và tên: Đào Trọng Nguyên

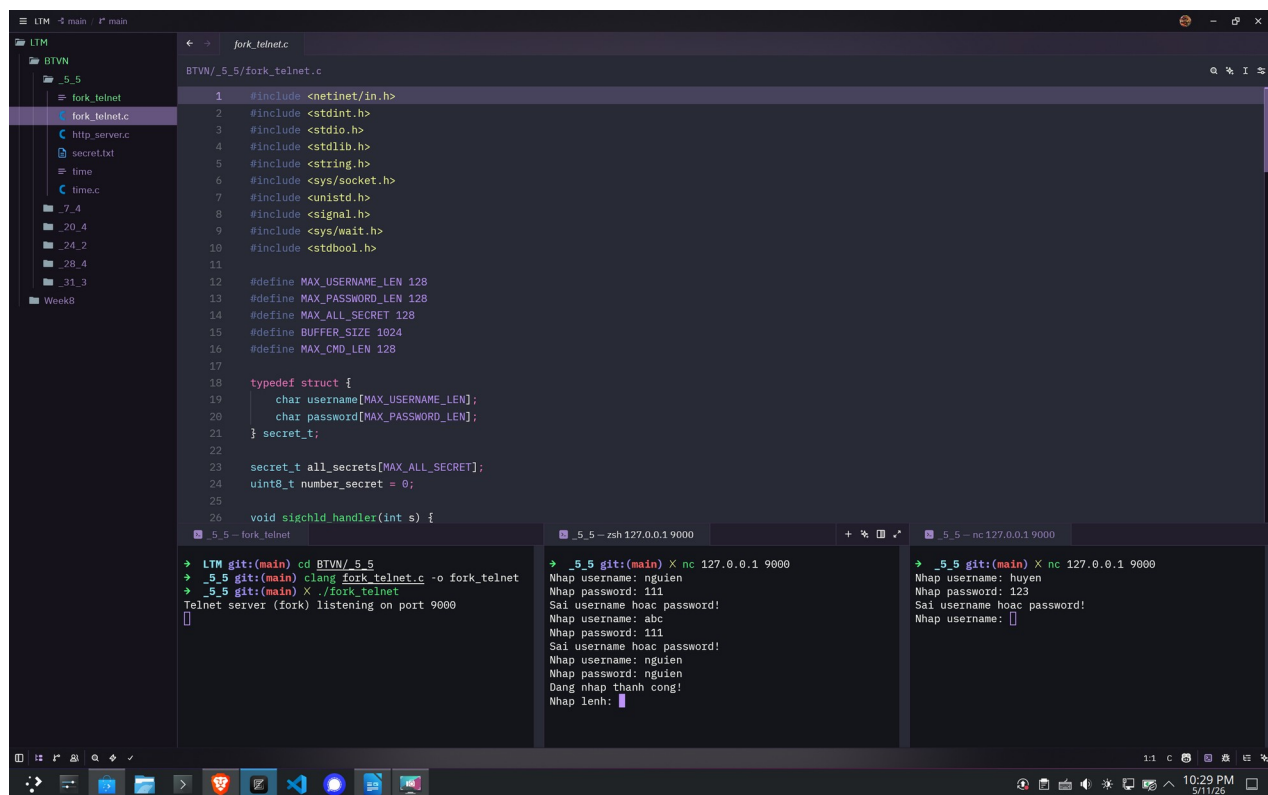
MSSV: 20235390

Ngành: Kỹ thuật máy tính -K68

Mã lớp: 168503

Báo cáo bài tập về nhà

Bài 1: Lập trình lại ứng dụng telnet_server (slide 174) sử dụng kỹ thuật multiprocessing.



The screenshot displays a code editor with the file `fork_telnet.c` open. The code includes headers for `<netinet/in.h>`, `<stdint.h>`, `<stdio.h>`, `<stdlib.h>`, `<string.h>`, `<sys/socket.h>`, `<unistd.h>`, `<signal.h>`, `<sys/wait.h>`, and `<stdbool.h>`. It defines constants for `MAX_USERNAME_LEN`, `MAX_PASSWORD_LEN`, `MAX_ALL_SECRET`, `BUFFER_SIZE`, and `MAX_CMD_LEN`. A `secret_t` struct is defined with `username` and `password` arrays. The `sigchld_handler` function is also shown. Below the code, three terminal windows are open, showing the compilation and execution of the program. The first terminal shows the compilation command `clang fork_telnet.c -o fork_telnet` and the output `Telnet server (fork) listening on port 9000`. The second terminal shows the execution of the server with `nc 127.0.0.1 9000`, displaying the login process for a user named 'nguien' with password '111'. The third terminal shows the execution of the server with `nc 127.0.0.1 9000`, displaying the login process for a user named 'huyen' with password '123'.

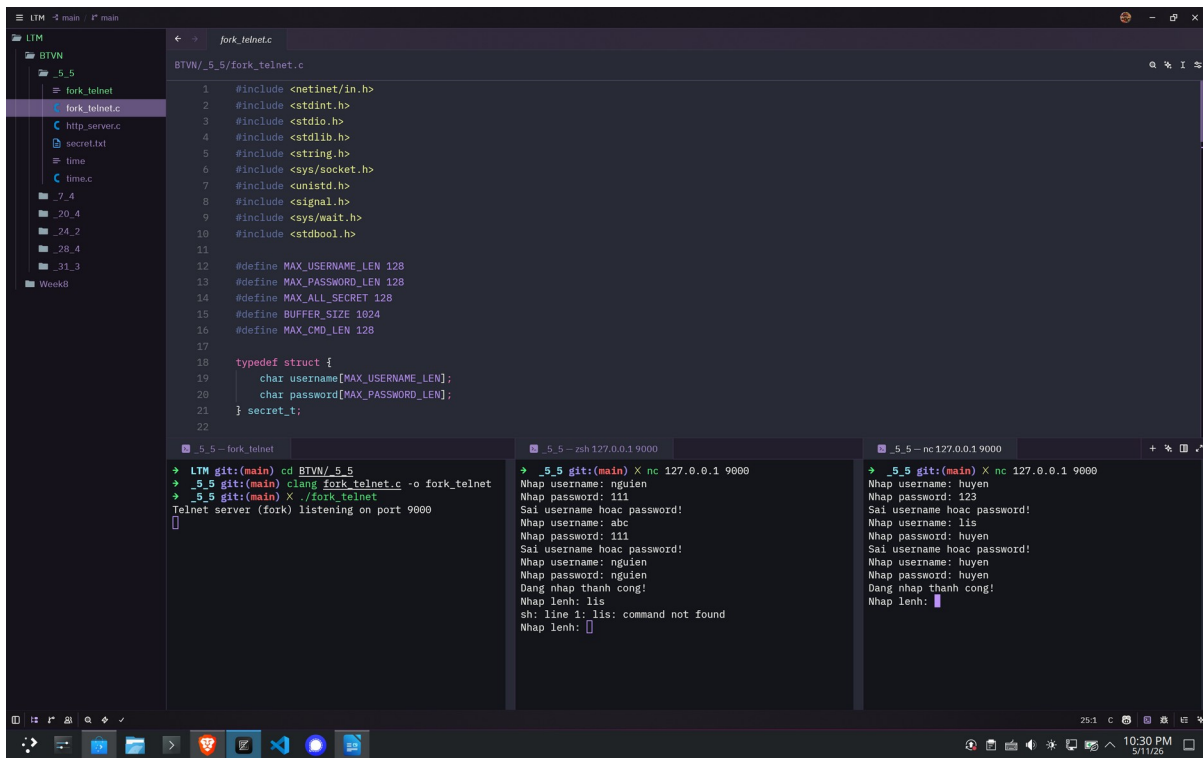
```
1 #include <netinet/in.h>
2 #include <stdint.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <string.h>
6 #include <sys/socket.h>
7 #include <unistd.h>
8 #include <signal.h>
9 #include <sys/wait.h>
10 #include <stdbool.h>
11
12 #define MAX_USERNAME_LEN 128
13 #define MAX_PASSWORD_LEN 128
14 #define MAX_ALL_SECRET 128
15 #define BUFFER_SIZE 1024
16 #define MAX_CMD_LEN 128
17
18 typedef struct {
19     char username[MAX_USERNAME_LEN];
20     char password[MAX_PASSWORD_LEN];
21 } secret_t;
22
23 secret_t all_secrets[MAX_ALL_SECRET];
24 uint8_t number_secret = 0;
25
26 void sigchld_handler(int s) {
```

```
LTM git:(main) cd BTVN/_5_5
→ _5_5 git:(main) clang fork_telnet.c -o fork_telnet
→ _5_5 git:(main) ./fork_telnet
Telnet server (fork) listening on port 9000

→ _5_5 git:(main) X nc 127.0.0.1 9000
Nhap username: nguien
Nhap password: 111
Sai username hoặc password!
Nhap username: abc
Nhap password: 111
Sai username hoặc password!
Nhap username: nguien
Nhap password: nguien
Dang nhap thanh cong!
Nhap lenh:

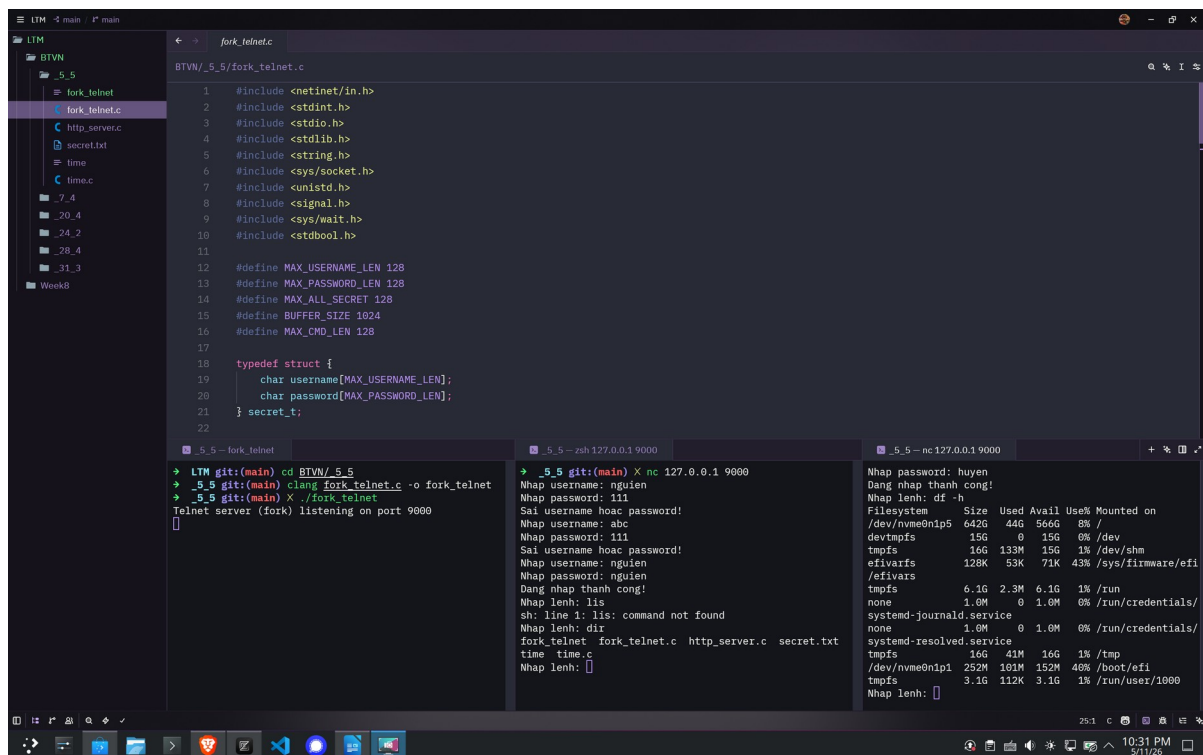
→ _5_5 git:(main) X nc 127.0.0.1 9000
Nhap username: huyen
Nhap password: 123
Sai username hoặc password!
Nhap username:
```

Ảnh 1- Testcase khi đăng nhập đúng và sai



```
1 #include <netinet/in.h>
2 #include <stdint.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <string.h>
6 #include <sys/socket.h>
7 #include <unistd.h>
8 #include <signal.h>
9 #include <sys/wait.h>
10 #include <stdbool.h>
11
12 #define MAX_USERNAME_LEN 128
13 #define MAX_PASSWORD_LEN 128
14 #define MAX_ALL_SECRET 128
15 #define BUFFER_SIZE 1024
16 #define MAX_CMD_LEN 128
17
18 typedef struct {
19     char username[MAX_USERNAME_LEN];
20     char password[MAX_PASSWORD_LEN];
21 } secret_t;
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

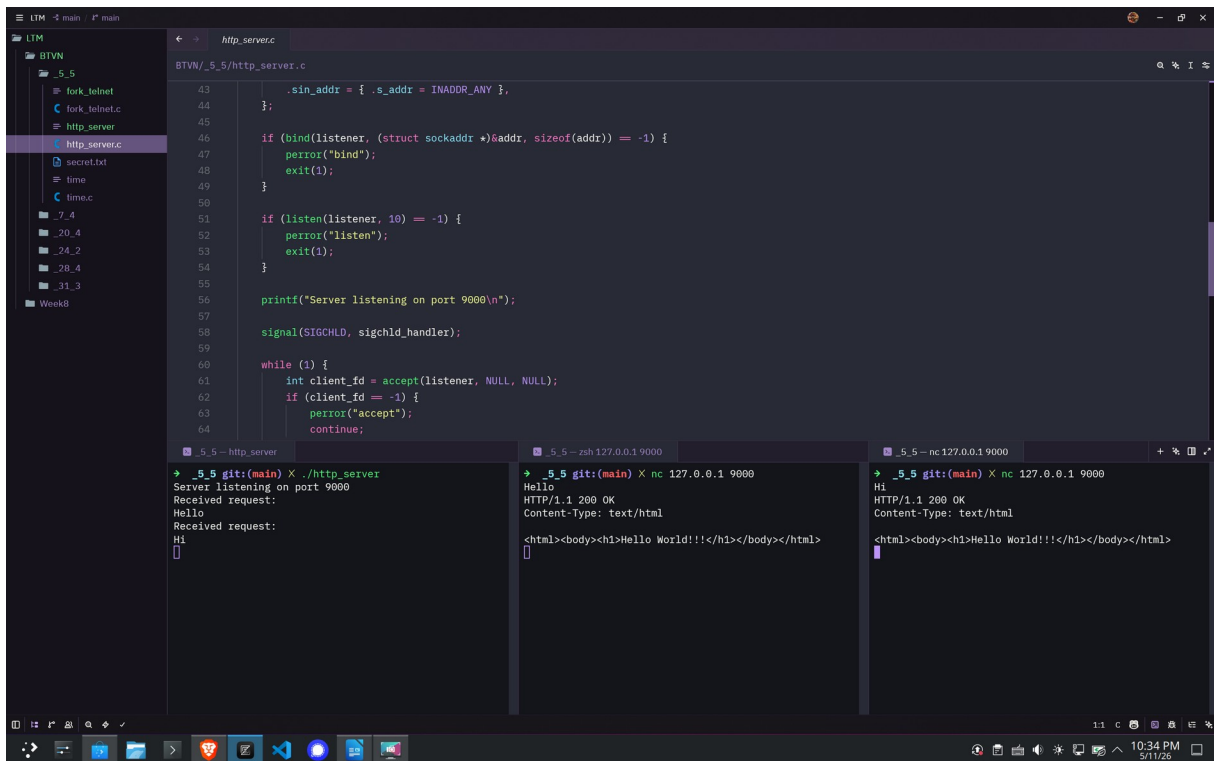
Ảnh 2- Testcase khi nhập lệnh sai



```
1 #include <netinet/in.h>
2 #include <stdint.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <string.h>
6 #include <sys/socket.h>
7 #include <unistd.h>
8 #include <signal.h>
9 #include <sys/wait.h>
10 #include <stdbool.h>
11
12 #define MAX_USERNAME_LEN 128
13 #define MAX_PASSWORD_LEN 128
14 #define MAX_ALL_SECRET 128
15 #define BUFFER_SIZE 1024
16 #define MAX_CMD_LEN 128
17
18 typedef struct {
19     char username[MAX_USERNAME_LEN];
20     char password[MAX_PASSWORD_LEN];
21 } secret_t;
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Ảnh 3- Testcase hiện kết quả lệnh “dir” và “df-h”

Bài 2: Lập trình lại ứng dụng http_server (slide 154) sử dụng kỹ thuật preforking.



```
43 .sin_addr = { .s_addr = INADDR_ANY },
44 };
45
46 if (bind(listener, (struct sockaddr *)&addr, sizeof(addr)) == -1) {
47     perror("bind");
48     exit(1);
49 }
50
51 if (listen(listener, 10) == -1) {
52     perror("listen");
53     exit(1);
54 }
55
56 printf("Server listening on port 9000\n");
57
58 signal(SIGCHLD, sigchld_handler);
59
60 while (1) {
61     int client_fd = accept(listener, NULL, NULL);
62     if (client_fd == -1) {
63         perror("accept");
64         continue;
65     }
66     fork();
67     // ... (rest of the code) ...
68 }
```

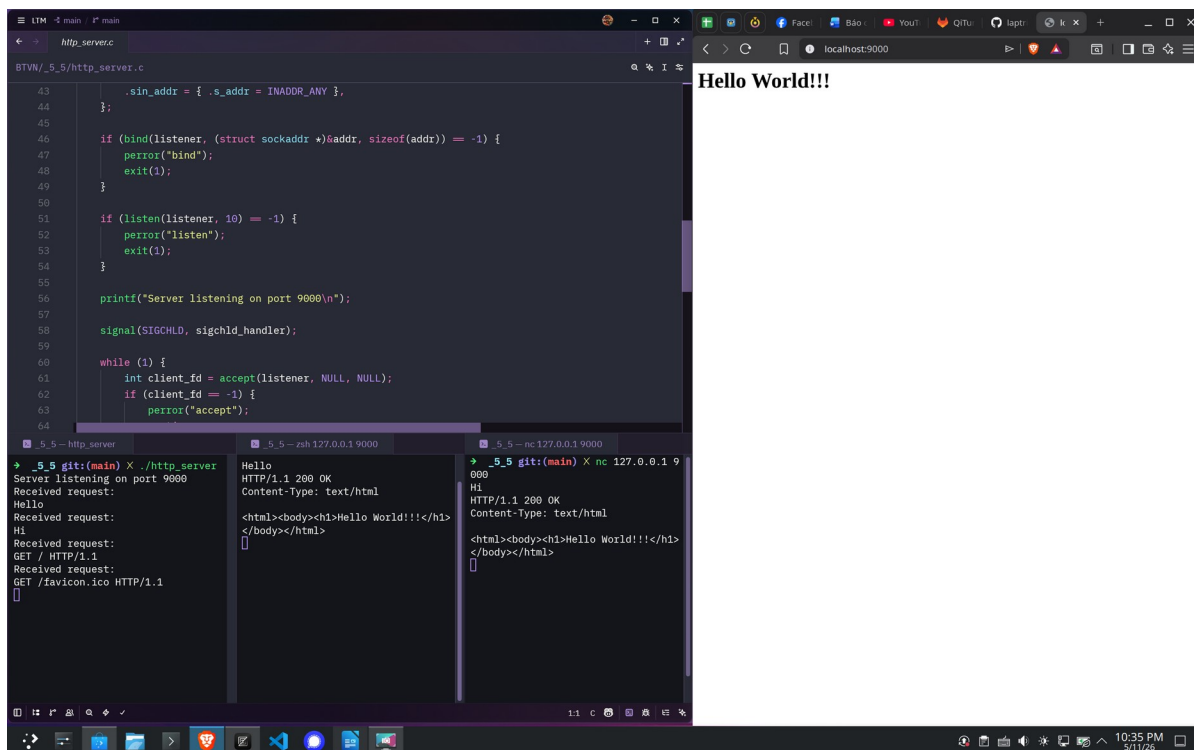
Terminal output:

```
_5_5 - http_server
Server listening on port 9000
Received request:
Hello
Received request:
Hi
[ ]

_5_5 - zsh 127.0.0.1 9000
Hello
HTTP/1.1 200 OK
Content-Type: text/html
<html><body><h1>Hello World!!!</h1></body></html>
[ ]

_5_5 - nc 127.0.0.1 9000
Hi
HTTP/1.1 200 OK
Content-Type: text/html
<html><body><h1>Hello World!!!</h1></body></html>
[ ]
```

Ảnh 4- Testcase gửi dữ liệu và hiển thị kết quả server gửi lên client



```
43 .sin_addr = { .s_addr = INADDR_ANY },
44 };
45
46 if (bind(listener, (struct sockaddr *)&addr, sizeof(addr)) == -1) {
47     perror("bind");
48     exit(1);
49 }
50
51 if (listen(listener, 10) == -1) {
52     perror("listen");
53     exit(1);
54 }
55
56 printf("Server listening on port 9000\n");
57
58 signal(SIGCHLD, sigchld_handler);
59
60 while (1) {
61     int client_fd = accept(listener, NULL, NULL);
62     if (client_fd == -1) {
63         perror("accept");
64         continue;
65     }
66     fork();
67     // ... (rest of the code) ...
68 }
```

Terminal output:

```
_5_5 git:(main) X ./http_server
Server listening on port 9000
Received request:
Hello
Received request:
Hi
Received request:
GET / HTTP/1.1
Received request:
GET /favicon.ico HTTP/1.1
[ ]

_5_5 - zsh 127.0.0.1 9000
Hello
HTTP/1.1 200 OK
Content-Type: text/html
<html><body><h1>Hello World!!!</h1></body></html>
[ ]

_5_5 - nc 127.0.0.1 9000
Hi
HTTP/1.1 200 OK
Content-Type: text/html
<html><body><h1>Hello World!!!</h1></body></html>
[ ]
```

Ảnh 5- Kết quả server hiển thị trên website

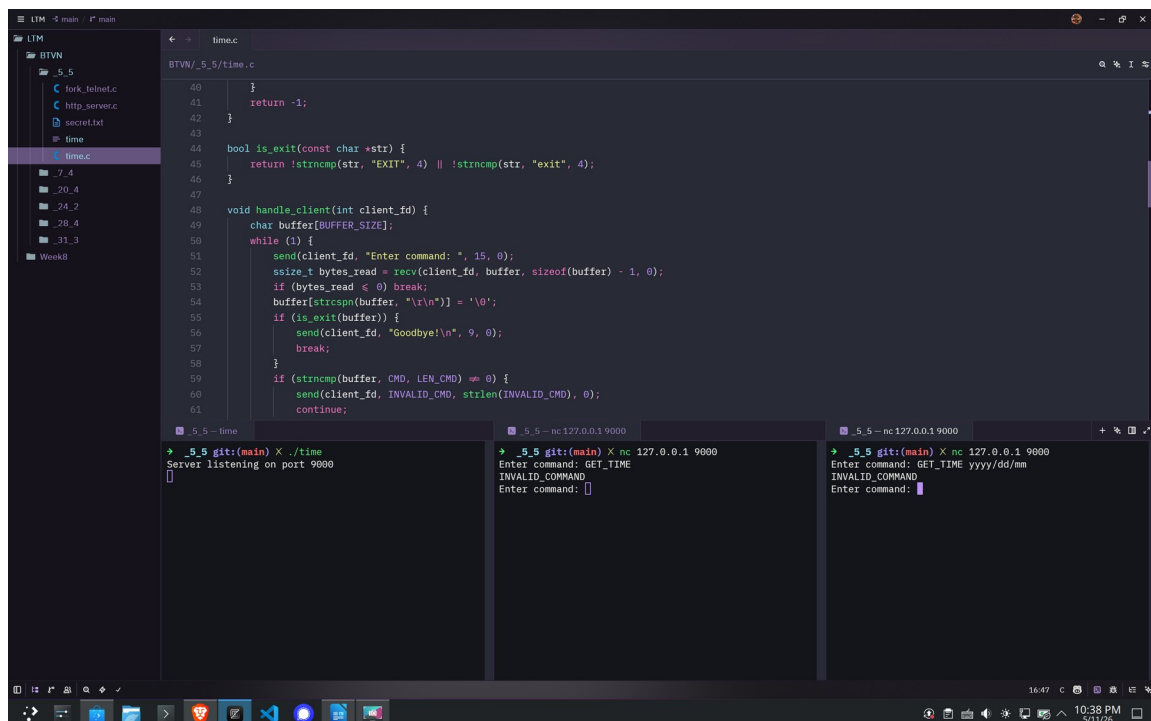
Bài 3:

- Lập trình ứng dụng time_server thực hiện chức năng sau:
- Chấp nhận nhiều kết nối từ các client sử dụng kỹ thuật multiprocessing.
- Client gửi lệnh:

GET_TIME [format]

để nhận thời gian từ server.

- format là định dạng thời gian server cần trả về client.
- Các format cần hỗ trợ:
- dd/mm/yyyy → ví dụ: 30/01/2023
- dd/mm/yy → ví dụ: 30/01/23
- mm/dd/yyyy → ví dụ: 01/30/2023
- mm/dd/yy → ví dụ: 01/30/23
- Cần kiểm tra tính đúng đắn của lệnh client gửi lên server.



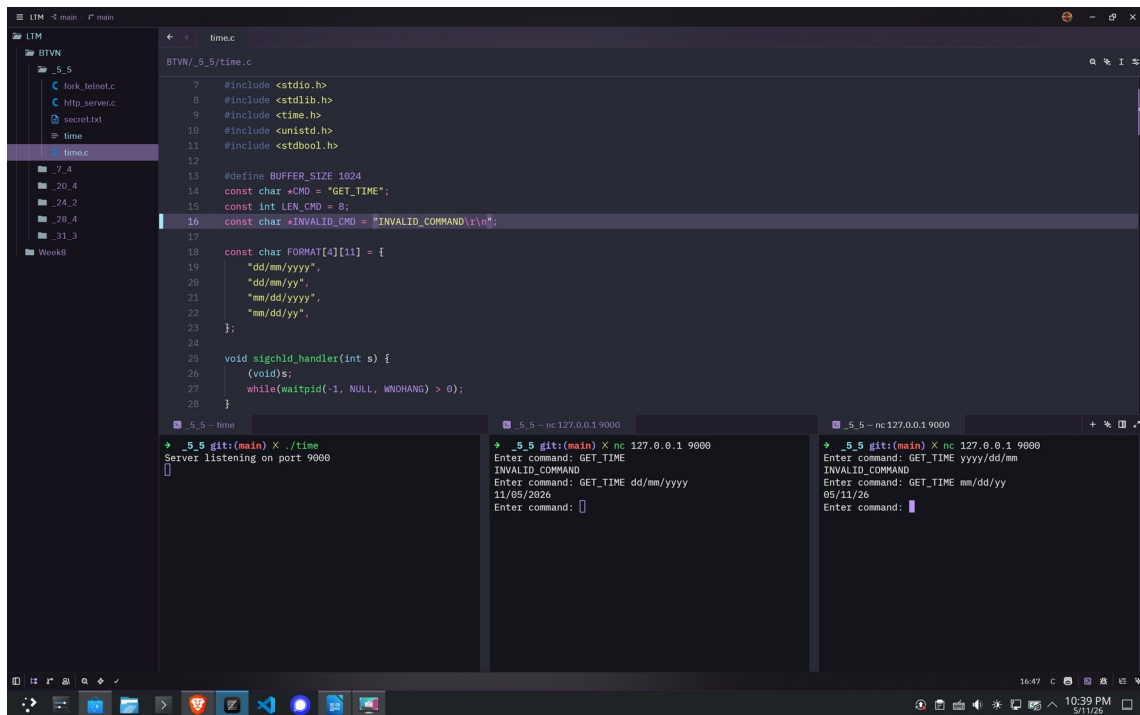
```
40 }
41 return -1;
42 }
43
44 bool is_exit(const char *str) {
45     return !strcmp(str, "EXIT", 4) || !strcmp(str, "exit", 4);
46 }
47
48 void handle_client(int client_fd) {
49     char buffer[BUFFER_SIZE];
50     while (1) {
51         send(client_fd, "Enter command: ", 15, 0);
52         ssize_t bytes_read = recv(client_fd, buffer, sizeof(buffer) - 1, 0);
53         if (bytes_read <= 0) break;
54         buffer[strlen(buffer)] = '\0';
55         if (is_exit(buffer)) {
56             send(client_fd, "Goodbye!\n", 9, 0);
57             break;
58         }
59         if (strcmp(buffer, CMD, LEN_CMD) == 0) {
60             send(client_fd, INVALID_CMD, strlen(INVALID_CMD), 0);
61             continue;
62         }
63     }
64 }
```

5.5 - time
Server listening on port 9000

5.5 - nc 127.0.0.1 9000
Enter command: GET_TIME
Enter command: []

5.5 - nc 127.0.0.1 9000
Enter command: GET_TIME yyyy/dd/mm
INVALID_COMMAND
Enter command: []

Ảnh 6- Nhập sai lệnh và sai format



```
BTWN/_5_5/time.c
7  #include <stdio.h>
8  #include <stdlib.h>
9  #include <time.h>
10 #include <unistd.h>
11 #include <stdbool.h>
12
13 #define BUFFER_SIZE 1024
14 const char *CMD = "GET_TIME";
15 const int LEN_CMD = 8;
16 const char *INVALID_CMD = "INVALID_COMMAND\n";
17
18 const char FORMAT[4][11] = {
19     "dd/mm/yyyy",
20     "dd/mm/yy",
21     "mm/dd/yyyy",
22     "mm/dd/yy",
23 };
24
25 void sigchld_handler(int s) {
26     (void)s;
27     while(waitpid(-1, NULL, WNOHANG) > 0);
28 }

```

Terminal 1: `_5_5 git:(main) X ./time`
Server listening on port 9000

Terminal 2: `_5_5 - nc 127.0.0.1 9000`
Enter command: GET_TIME
INVALID_COMMAND
Enter command: GET_TIME dd/mm/yyyy
11/05/2020
Enter command:

Terminal 3: `_5_5 - nc 127.0.0.1 9000`
Enter command: GET_TIME yyyy/dd/mm
INVALID_COMMAND
Enter command: GET_TIME mm/dd/yy
05/11/20
Enter command:

Ảnh 7- Server gửi kết quả lệnh cho client